

# Parcial 1 — Paradigmas y Lenguajes de Programación 2026

UNTDF — Instituto IDEI

25 de mayo de 2026

## PARCIAL 1 — Paradigmas y Lenguajes de Programación

**Materia:** Paradigmas y Lenguajes de Programación 2026

**Institución:** UNTDF — Instituto IDEI

**Docente:** Matías Gel

**Modalidad:** Quiz en clase | **Duración:** 60 minutos

**Tipo:** Opción múltiple (A / B / C / D)

---

**Nombre:** \_\_\_\_\_

**Legajo:** \_\_\_\_\_

**Fecha:** 25/05/2026

---

**Instrucciones:** Marcá con una X la opción correcta en cada pregunta. Una sola respuesta por pregunta. Puntaje máximo: 100 pts. Aprobación:  $\geq 60$  pts.

---

### Tema 02 — Sintaxis y Semántica

1. (3 pts) ¿Cuál de las siguientes afirmaciones describe correctamente la diferencia entre sintaxis y semántica de un lenguaje de programación?

- ☐ A) La sintaxis define el significado de los programas; la semántica define su forma correcta
- ☐ B) La sintaxis define las reglas de forma que determinan si un programa está bien construido; la semántica define el significado de los programas
- ☐ C) La sintaxis y la semántica son equivalentes: un error en una implica un error en la otra
- ☐ D) La semántica es responsabilidad del programador; la sintaxis es responsabilidad del compilador

---

2. (4 pts) Considerá el siguiente programa TypeScript:

```
function suma(a: number, b: number): number {  
  return a + b;  
}
```

```
const resultado = suma(10, "hola");
```

¿En qué nivel es incorrecto este programa y por qué?

- ☐ A) Es incorrecto sintácticamente: la llamada `suma(10, "hola")` no respeta la gramática de TypeScript
  - ☐ B) Es incorrecto semánticamente: pasa el análisis de forma, pero viola las reglas de tipo al pasar un `string` donde se espera un `number`
  - ☐ C) Es correcto tanto sintáctica como semánticamente: TypeScript acepta cualquier valor en una llamada a función
  - ☐ D) Es incorrecto léxicamente: el string `"hola"` no es un token válido en TypeScript
- 

## Tema 03 — Intro al Paradigma Funcional

3. (3 pts) ¿Cuáles son los tres pilares del paradigma de programación funcional?

- ☐ A) Herencia, polimorfismo y encapsulamiento
  - ☐ B) Secuencia, selección e iteración
  - ☐ C) Funciones puras, inmutabilidad y transparencia referencial
  - ☐ D) Abstracción, modularidad y encapsulamiento
- 

4. (3 pts) El lambda-cálculo de Alonzo Church y la Máquina de Turing de Alan Turing (ambos de 1936) demostraron ser equivalentes en poder computacional. Sin embargo, representan modelos conceptualmente distintos de qué es “computar”. ¿Cuál es la diferencia fundamental entre ambos modelos?

- ☐ A) La Máquina de Turing opera sobre números enteros; el lambda-cálculo opera sobre funciones de orden superior
  - ☐ B) La Máquina de Turing define la computación como modificación secuencial de estado (cinta); el lambda-cálculo define la computación como sustitución/reescritura de expresiones
  - ☐ C) La Máquina de Turing es determinista; el lambda-cálculo es no determinista
  - ☐ D) El lambda-cálculo requiere hardware especializado; la Máquina de Turing se puede ejecutar en cualquier computadora
- 

5. (3 pts) ¿Qué significa que una función tiene “transparencia referencial”?

- ☐ A) Que la función puede recibir otras funciones como argumentos
- ☐ B) Que el nombre de la función refleja claramente lo que hace
- ☐ C) Que la función puede ser reemplazada por su valor de retorno en cualquier parte del código sin cambiar el comportamiento del programa
- ☐ D) Que la función no usa variables globales pero sí puede modificar sus argumentos

---

6. (4 pts) Considerá el siguiente código TypeScript en estilo funcional:

```
const numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
const resultado = numeros
  .filter(n => n % 2 === 0)
  .map(n => n * n);
```

¿Qué contiene resultado después de ejecutar este código?

- ☐ A) [1, 4, 9, 16, 25, 36, 49, 64, 81, 100] — los cuadrados de todos los números
- ☐ B) [2, 4, 6, 8, 10] — los números pares del arreglo original
- ☐ C) [4, 16, 36, 64, 100] — los cuadrados de los números pares
- ☐ D) [1, 3, 5, 7, 9] — los números impares del arreglo original

---

7. (4 pts) Considerá las siguientes dos funciones en TypeScript:

```
// Función A
let contador = 0;
const incrementarA = (x: number): number => {
  contador++;
  return x + 1;
};
```

```
// Función B
const incrementarB = (x: number): number => {
  return x + 1;
};
```

¿Cuál de las siguientes afirmaciones es correcta respecto a la pureza de estas funciones?

- ☐ A) Ambas son funciones puras porque las dos devuelven  $x + 1$
- ☐ B) `incrementarA` es pura porque su resultado  $x + 1$  no depende de `contador`; `incrementarB` también es pura
- ☐ C) `incrementarA` no es una función pura porque modifica la variable externa `contador`, produciendo un efecto colateral; `incrementarB` sí es pura
- ☐ D) Ninguna es pura porque TypeScript no soporta funciones puras nativas

---

## Tema 04 — Funcional Avanzado

8. (3 pts) ¿Qué es una función de orden superior (Higher Order Function — HOF)?

- ☐ A) Una función que solo opera sobre tipos numéricos

- ☐ B) Una función que recibe otra función como argumento y/o devuelve una función como resultado
  - ☐ C) Una función que utiliza recursión para resolver un problema
  - ☐ D) Una función declarada con `function` en lugar de arrow function en TypeScript
- 

9. (3 pts) ¿Cuál es la diferencia fundamental entre **partial application** y **currying**?

- ☐ A) Partial application convierte una función en una función currificada; currying aplica parcialmente sus argumentos
  - ☐ B) Partial application fija uno o más argumentos de una función devolviendo una función con menos parámetros; currying transforma una función de N argumentos en una cadena de N funciones de un argumento cada una
  - ☐ C) Son el mismo concepto expresado en distintos lenguajes: partial en Clojure y currying en TypeScript
  - ☐ D) Currying siempre requiere especificar todos los argumentos de una vez; partial application permite aplicarlos en cualquier orden
- 

10. (3 pts) `compose(f, g)(x)` y `pipe(f, g)(x)` aplican las mismas funciones `f` y `g` sobre `x`, pero en distinto orden. ¿Cuál de las siguientes afirmaciones es correcta?

- ☐ A) `compose(f, g)(x)` aplica `f` primero y luego `g`; `pipe(f, g)(x)` aplica `g` primero y luego `f`
  - ☐ B) `compose(f, g)(x)` es equivalente a `f(g(x))` — se aplica de derecha a izquierda; `pipe(f, g)(x)` es equivalente a `g(f(x))` — se aplica de izquierda a derecha
  - ☐ C) Ambos producen el mismo resultado porque la composición es conmutativa
  - ☐ D) `compose` solo existe en Clojure; `pipe` solo existe en TypeScript
- 

11. (4 pts) Dado el siguiente código TypeScript:

```
const trim = (s: string): string => s.trim();
const toLower = (s: string): string => s.toLowerCase();
const addDomain = (s: string): string => s + "@untdf.edu.ar";

const pipe = <T>(...fns: Array<(x: T) => T>) => (x: T) =>
  fns.reduce((acc, fn) => fn(acc), x);

const normalizeEmail = pipe(trim, toLower, addDomain);

console.log(normalizeEmail(" MATIAS "));
```

¿Qué imprime este código?

- ☐ A) " MATIAS @untdf.edu.ar" — pipe no aplica trim porque está primero

- ☐ B) "matias@untdf.edu.ar" — se aplican trim, luego toLower, luego addDomain en orden
  - ☐ C) "MATIAS@UNTDF.EDU.AR" — toLower se aplica solo al resultado de trim, no a addDomain
  - ☐ D) Error de compilación — pipe no acepta funciones de tipo string => string
- 

12. (4 pts) Considerá esta implementación de factorial en TypeScript:

*// Versión A*

```
const factorialA = (n: number): number => {  
  if (n <= 1) return 1;  
  return n * factorialA(n - 1);  
};
```

*// Versión B*

```
const factorialB = (n: number, acc: number = 1): number => {  
  if (n <= 1) return acc;  
  return factorialB(n - 1, n * acc);  
};
```

¿Por qué factorialB es preferible a factorialA desde la perspectiva del paradigma funcional?

- ☐ A) factorialB es más legible porque usa un parámetro con valor por defecto
  - ☐ B) factorialA usará menos memoria porque no necesita el acumulador; factorialB es más lenta
  - ☐ C) factorialB implementa recursión de cola: la llamada recursiva es la última operación, lo que permite que el runtime optimice la pila (TCO); factorialA acumula marcos de stack porque necesita multiplicar por n al regresar
  - ☐ D) Ambas son equivalentes en términos de eficiencia — TypeScript optimiza la recursión automáticamente
- 

## Tema 06 — Paradigma Lógico: Prolog

13. (3 pts) ¿Cuáles son los tres tipos de enunciados que forman un programa Prolog?

- ☐ A) Variables, predicados y términos
  - ☐ B) Hechos, reglas y consultas
  - ☐ C) Funciones, clases y módulos
  - ☐ D) Proposiciones, implicaciones y conjunciones
- 

14. (3 pts) ¿Cuál es la diferencia fundamental en la forma de “programar” entre el paradigma imperativo y el paradigma lógico?

- ☐ A) En el imperativo se usa `if-else`; en el lógico se usa `case-when`
  - ☐ B) En el imperativo se describe el conocimiento del dominio; en el lógico se describe el algoritmo de resolución
  - ☐ C) En el imperativo se especifica cómo resolver el problema (algoritmo); en el lógico se especifica qué es verdadero (conocimiento) y el motor de inferencia encuentra las soluciones
  - ☐ D) El paradigma lógico solo funciona para problemas matemáticos; el imperativo es de propósito general
- 

**15. (3 pts) En Prolog, ¿qué representa una variable y cómo se diferencia de una constante en la sintaxis del lenguaje?**

- ☐ A) Las variables comienzan con minúscula (ej: `ana`); las constantes comienzan con mayúscula (ej: `X`)
  - ☐ B) Las variables comienzan con mayúscula o `_` (ej: `X`, `_Y`); las constantes (átomos) comienzan con minúscula (ej: `ana`, `carlos`)
  - ☐ C) No hay variables en Prolog; todo son constantes simbólicas
  - ☐ D) Las variables se declaran con `var`; las constantes con `const`, igual que en JavaScript
- 

**16. (4 pts) Dada la siguiente base de conocimiento Prolog:**

```
padre(carlos, laura).
padre(carlos, pedro).
padre(tomas, carlos).
padre(tomas, beatriz).
```

```
abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
```

**¿Cuál es el resultado de la consulta `?- abuelo(tomas, Z).`?**

- ☐ A) `false` — no hay suficiente información en la base
  - ☐ B) `Z = carlos` — Tomás es abuelo de Carlos
  - ☐ C) `Z = laura ; Z = pedro` — Tomás es abuelo de Laura y Pedro
  - ☐ D) `Z = carlos ; Z = beatriz` — Tomás es padre de Carlos y Beatriz, no abuelo
- 

**17. (4 pts) Considera la siguiente regla Prolog:**

```
hermano(X, Y) :- padre(P, X), padre(P, Y), X \= Y.
```

**¿Cuál de las siguientes afirmaciones describe correctamente lo que hace esta regla?**

- ☐ A) Define que X es hermano de Y si X y Y tienen el mismo padre P y X es distinto de Y
- ☐ B) Define que X es hermano de Y si X es padre de P y P es padre de Y
- ☐ C) Define que X es hermano de Y solo si ambos tienen exactamente un padre en común

- ☐ D) La regla tiene un error: en Prolog no se puede usar  $\backslash=$  para comparar variables
- 

## Tema 07 — Lógico Avanzado: Unificación y Backtracking

### 18. (3 pts) ¿Qué es la unificación en Prolog?

- ☐ A) Un algoritmo de búsqueda que recorre el árbol de soluciones de izquierda a derecha
- ☐ B) El proceso de calcular el valor numérico de una expresión aritmética
- ☐ C) El proceso de encontrar una sustitución de variables que haga que dos términos sean sintácticamente idénticos
- ☐ D) La técnica de ordenar los hechos en la base de conocimiento para optimizar las consultas
- 

### 19. (3 pts) En Prolog, si ejecutamos $?- X = 2 + 3.$ , el motor responde $X = 2+3$ sin evaluar la suma. ¿Por qué?

- ☐ A) Porque Prolog no tiene operadores aritméticos
- ☐ B) Porque  $=$  realiza unificación, no evaluación.  $2+3$  es una estructura  $+(2,3)$  que se unifica como término; para evaluar numéricamente se usa  $is/2$
- ☐ C) Porque  $X$  es una variable entera y no puede contener una expresión
- ☐ D) Porque la unificación en Prolog es lazy y solo evalúa cuando se necesita el resultado
- 

### 20. (3 pts) ¿Qué es un “choice point” (punto de elección) en el backtracking de Prolog?

- ☐ A) El punto del programa donde el programador declara que hay una sola solución posible
- ☐ B) Una marca que el motor crea cuando encuentra múltiples cláusulas que pueden unificar con el goal actual, guardando las alternativas para explorar si la rama actual falla
- ☐ C) Un predicado especial que le permite al programador elegir entre varias bases de conocimiento
- ☐ D) El nodo raíz del árbol SLD a partir del cual se inicia la resolución
- 

### 21. (3 pts) ¿Cuál es la diferencia entre $=$ y $==$ en Prolog?

- ☐ A)  $=$  compara por valor;  $==$  compara por referencia de memoria
- ☐ B)  $=$  intenta unificar los dos términos (puede ligar variables);  $==$  verifica si los dos términos son ya idénticos sin ligar ninguna variable
- ☐ C)  $=$  solo funciona con átomos;  $==$  funciona con cualquier término
- ☐ D) Son sinónimos — en SWI-Prolog ambos realizan la misma operación
- 

### 22. (3 pts) ¿Cuál de las siguientes consultas de unificación tiene ÉXITO en Prolog?

- ☐ A)  $?- f(X, b) = f(a, a).$
- ☐ B)  $?- f(X, X) = f(ana, beatriz).$

- ☐ C)  $?- f(X, b) = f(a, Y).$
  - ☐ D)  $?- f(X, g(X)) = f(a, g(b)).$
- 

23. (4 pts) Dada la siguiente base de conocimiento:

```
miembro(X, [X|_]).
miembro(X, [_|T]) :- miembro(X, T).
```

¿Cuáles son TODAS las respuestas que da Prolog a  $?- \text{miembro}(X, [a, b, c]).?$

- ☐ A) Solo  $X = a$  — Prolog devuelve la primera solución encontrada y se detiene
  - ☐ B)  $X = a$  ;  $X = b$  ;  $X = c$  — Prolog devuelve las tres soluciones vía backtracking
  - ☐ C)  $X = [a, b, c]$  — Prolog unifica  $X$  con la lista completa
  - ☐ D) `false` — `miembro/2` requiere que el primer argumento ya esté instanciado
- 

24. (4 pts) Considera el siguiente programa:

```
color(rojo). color(verde). color(azul).

?- color(X), write(X), nl, fail.
```

¿Qué imprime esta consulta y por qué?

- ☐ A) Solo rojo — porque `fail` hace que la consulta falle después del primer resultado
  - ☐ B) No imprime nada — `fail` hace que toda la consulta falle antes de ejecutar `write`
  - ☐ C) Imprime rojo, luego verde, luego azul, luego falla — `fail` fuerza el backtracking que agota todos los valores de `color/1`
  - ☐ D) Imprime rojo verde azul en una sola línea separado por espacios
- 

25. (4 pts) Considera los siguientes dos predicados:

```
% Versión A
maximo(X, Y, X) :- X >= Y.
maximo(X, Y, Y) :- X < Y.
```

```
% Versión B
maximo(X, Y, X) :- X >= Y, !.
maximo(_, Y, Y).
```

¿Cuál es la diferencia de comportamiento entre la Versión A y la Versión B?

- ☐ A) Ambas son equivalentes en todos los casos — el corte `!` es solo una optimización de rendimiento sin impacto semántico



- ☐ **B)** La Versión B usa corte **verde**: elimina elección cuando ya se sabe cuál es el máximo, sin cambiar las soluciones; la Versión A recalcula ambas condiciones; ambas dan el mismo resultado para enteros instanciados
  - ☐ **C)** La Versión B usa corte **rojo**: si se llama con variables no instanciadas, el corte hace que se devuelva una sola solución posiblemente incorrecta; la Versión A se comporta correctamente en más contextos
  - ☐ **D)** La Versión A falla si X e Y son iguales; la Versión B siempre devuelve X en ese caso
- 

**26. (3 pts) El corte ! en Prolog fue descrito en clase como una “navaja de doble filo”. ¿Cuál de las siguientes afirmaciones captura mejor esta idea?**

- ☐ **A)** El corte es siempre perjudicial y no debería usarse en programas bien diseñados
  - ☐ **B)** El corte verde mejora la eficiencia sin cambiar las soluciones; el corte rojo cambia el significado declarativo del programa, haciendo que Prolog encuentre menos soluciones de las que debería encontrar lógicamente
  - ☐ **C)** El corte siempre mejora la eficiencia porque poda el árbol de búsqueda, pero nunca afecta la corrección del programa
  - ☐ **D)** El corte rojo y el verde son sinónimos — la diferencia es solo de estilo
- 

## **Tema 08 — Paradigma OO con TypeScript**

**27. (3 pts) ¿Cuáles son los cuatro pilares del paradigma orientado a objetos?**

- ☐ **A)** Secuencia, selección, iteración y recursión
  - ☐ **B)** Encapsulamiento, abstracción, herencia y polimorfismo
  - ☐ **C)** Funciones puras, inmutabilidad, composición y transparencia referencial
  - ☐ **D)** Tipos, clases, interfaces y módulos
- 

**28. (3 pts) Smalltalk afirma que “todo es un objeto” — incluyendo los números enteros y los booleanos. TypeScript no cumple este principio de la misma manera. ¿Por qué TypeScript no realiza el OO puro en el sentido de Smalltalk?**

- ☐ **A)** Porque TypeScript no tiene clases — solo tiene interfaces y tipos estructurales
  - ☐ **B)** Porque TypeScript tiene tipos primitivos (`number`, `boolean`, `string`) que no son objetos sino valores sin métodos propios, a diferencia de Smalltalk donde `1` y `true` son instancias de clases con métodos
  - ☐ **C)** Porque TypeScript fue diseñado como lenguaje funcional con sintaxis de clases opcional
  - ☐ **D)** Porque en TypeScript no existe el concepto de mensaje — las llamadas a método son solo invocaciones de función
-

29. (4 pts) Considerá la siguiente clase TypeScript:

```
class CuentaBancaria {  
  private saldo: number;  
  
  constructor(saldoInicial: number) {  
    this.saldo = saldoInicial;  
  }  
  
  depositar(monto: number): void {  
    this.saldo += monto;  
  }  
  
  getSaldo(): number {  
    return this.saldo;  
  }  
}
```

¿Cuál de los cuatro pilares del OO aplica PRINCIPALMENTE este diseño, y cómo?

- ☐ A) Herencia — la clase hereda de `Object` y extiende su comportamiento
  - ☐ B) Polimorfismo — `depositar` puede recibir distintos tipos de valores
  - ☐ C) Encapsulamiento — el campo `saldo` es `private`, accesible solo a través de los métodos públicos `depositar` y `getSaldo`
  - ☐ D) Abstracción — la clase oculta la implementación del sistema bancario completo
- 

30. (3 pts) En el paradigma funcional, la inmutabilidad es un principio central: los datos no se modifican. En el paradigma OO, el encapsulamiento gestiona el estado mutable. ¿Cuál de las siguientes afirmaciones describe mejor la diferencia en cómo cada paradigma trata el estado?

- ☐ A) No hay diferencia — ambos paradigmas prohíben la mutación del estado
  - ☐ B) El funcional elimina el estado mutable; el OO lo contiene dentro de los objetos y controla el acceso a él. El funcional gana en predecibilidad; el OO gana en capacidad de modelar entidades con identidad y ciclo de vida
  - ☐ C) El OO elimina el estado porque encapsular significa que el estado no existe fuera del objeto
  - ☐ D) El funcional permite mutación pero solo dentro de funciones; el OO permite mutación solo dentro de clases
-